

ONBOARDING · DESTRAVE CLAUDE CODE

O kit que faz o Claude Code virar *coluna vertebral*

Seis ferramentas que eu uso todo dia, com o prompt pronto pra você colar no seu Claude Code e deixar ele se instalar sozinho. Abra a página da skill, copia o bloco escuro inteiro, cola na sessão. Sem decoreba, sem tutorial em vídeo — o ambiente vira o meu.

O QUE ENTRA · 06 FERRAMENTAS

SKILL	CATEGORIA	PRA QUÊ
RTK	Dev essencial	Economia de 60-90% de token em operações repetidas (git, ls, etc.)
OpenWispr	Dev essencial	Dictation local — falar com o Claude em vez de digitar
agent-browser	Dev essencial	CLI de navegação web pro agente abrir páginas e ler conteúdo
Wiki (padrão)	Captura/memória	Segundo cérebro persistente em markdown, navegável no Obsidian
MemPalace	Captura/memória	Memória semântica entre conversas — KG + busca vetorial
/done	Ritual de fim	Fechar etapa, calibrar skills (auto-improvement) e fazer bridge com ClickUp

RTK — economia *silenciosa* de token

Proxy CLI escrito em Rust que intercepta comandos do Claude Code via hook e filtra output verboso antes de chegar no modelo. Você não muda nada na forma de trabalhar — só gasta 60-90% menos token em operações repetidas (git status, ls, find, etc.).

TIPO	PRÉ-REQUISITOS	QUANDO NÃO USAR
CLI binário + hook Claude Code	Cargo (Rust) ou Homebrew	Debug profundo de comando — use <code>rtk proxy <cmd></code>

PROMPT PRA COLAR NO CLAUDE CODE

Instale o RTK (Rust Token Killer) – proxy CLI que economiza token em comandos repetidos.

Passos:

1. Instale via cargo: ``cargo install rtk-cli`` (ou veja README do repo para opção brew se cargo não estiver disponível).
2. Verifique a instalação: rode ``rtk --version`` (deve mostrar versão) e ``rtk gain`` (deve abrir).
3. ⚠️ ATENÇÃO: existe outro pacote chamado ``rtk`` (reachingforthejack/rtk) – se ``rtk gain`` falhar com "command not found", o binário errado está no PATH. Desinstale o conflitante.
4. Configure o hook do Claude Code para reescrever automaticamente comandos comuns (git, ls, etc.) via ``rtk``. O setup do hook está documentado no README do repo `rtk-cli`.
5. Adicione ao meu CLAUDE.md global (~/.claude/CLAUDE.md) uma seção com os comandos meta: ``rtk gain``, ``rtk gain --history``, ``rtk discover``, ``rtk proxy``.
6. Confirme com ``rtk gain`` mostrando 0 economia (ainda não usei nada).

Se algum passo falhar, pause e me mostre o erro antes de continuar.

Verificação visual: depois de uns dias usando, rode `rtk gain` e veja a economia acumulada. Em sessões longas, é comum bater 70%+ em comandos de dev.

OpenWispr — você fala, o *Claude* entende

Dictation local rodando Whisper na sua máquina (sem nuvem, sem latência). Você segura **fn**, fala normalmente, solta — o texto aparece onde o cursor estiver. Plug direto no chat do Claude Code, em qualquer editor, em qualquer app. Resolve o gargalo de "tenho a ideia, mas digitar 3 parágrafos cansa".

TIPO	PRÉ-REQUISITOS	QUANDO NÃO USAR
Daemon CLI (macOS)	Homebrew · ~3GB livres pro modelo	Audio com música/barulho — qualidade cai

PROMPT PRA COLAR NO CLAUDE CODE

Instale o OpenWispr – dictation local com Whisper.

Passos:

1. Instale via Homebrew: ``brew install open-wispr`` (verifique o nome exato do tap; pode ser ``brew tap open-wispr/open-wispr`` primeiro).
2. Inicie o serviço: ``brew services start open-wispr``.
3. Verifique status: ``brew services list | grep open-wispr`` (deve mostrar started).
4. Baixe o modelo ``large-v3-turbo`` (rápido + bom em PT-BR). Caminho default: ``~/ .config/open-wispr/models/``. Se o download via app não funcionar, baixe manualmente o ``ggml-large-v3-turbo.bin`` da Hugging Face (ggerganov/whisper.cpp).
5. ⚠️ NÃO use o modelo ``large`` direto – URL canônico dele dá 404. Use ``large-v3-turbo``.
6. Teste: abra qualquer campo de texto, segure ``fn``, fale uma frase curta, solte. Texto deve aparecer.
7. Se o ícone na menu bar sumir atrás do notch, é normal – o daemon continua rodando.

Se a hotkey ``fn`` não funcionar, peça permissão de Acessibilidade em System Settings.

Por que importa: a maior parte do meu input pro Claude vem por voz. Sessão de 2h tem facilmente 30+ ditados longos. Sem isso, a fricção mata o uso.

agent-browser — o Claude com navegador próprio

CLI da Vercel Labs que dá ao Claude um browser headless com API simples. Diferente de WebFetch (que só puxa HTML), o agent-browser navega de verdade: clica, preenche form, lê screenshot. Refs @eN tornam a árvore amigável pro modelo. É a primeira escolha pra qualquer scraping ou navegação assistida.

TIPO	PRÉ-REQUISITOS	QUANDO NÃO USAR
CLI global Node.js	Node 20+ · npm/pnpm	API pública disponível — usa direto

PROMPT PRA COLAR NO CLAUDE CODE

Instale o agent-browser (Vercel Labs) – CLI de navegação web pro agente.

Passos:

- 1. Instale global: ``npm install -g @vercel/agent-browser`` (confirme o nome exato do pacote no README do repo vercel-labs/agent-browser).
- 2. Verifique: ``agent-browser --version``.
- 3. Faça um teste rápido: ``agent-browser snapshot https://example.com`` – deve devolver árvore com refs ``@eN``.
- 4. Adicione ao meu CLAUDE.md uma regra: para navegar web, hierarquia de preferência é (1) API oficial, (2) agent-browser, (3) playwright, (4) puppeteer. Não usar WebFetch quando precisar interagir com a página.
- 5. Confirme conhecendo os 3 verbos principais: ``snapshot`` (ler árvore), ``click @eN`` (clicar elemento), ``fill @eN "texto"`` (preencher).

Se Chrome/Chromium não estiver disponível, o agent-browser baixa o próprio na primeira run.

Caso real: "abre o dashboard X, copia o número de leads dessa semana e cola no relatório." Sem agent-browser, isso é manual ou exige scraping custom. Com ele, é uma instrução conversacional.

Wiki — seu segundo *cérebro* versionado

Não é uma skill instalável: é um padrão. Uma pasta `wiki/` no seu repo, mantida pelo próprio Claude, que vira fonte de verdade pra clientes, padrões e decisões. Navegável no Obsidian (grafo, backlinks, callouts) e indexável pelo modelo. O que importa é a estrutura — não a ferramenta.

TIPO	PRÉ-REQUISITOS	QUANDO NÃO USAR
Convenção de pastas + slash commands	Git repo · Obsidian (opcional)	Fragmentos soltos — vão pro MemPalace

PROMPT PRA COLAR NO CLAUDE CODE

Inicializa a estrutura de wiki no meu repo seguindo o padrão do Gabriel Pedrozo (PRODIA).

Passos:

1. Crie a pasta ``wiki/`` na raiz do projeto.
2. Dentro dela, crie subpastas: ``clientes/``, ``padroes/``, ``decisoese/``, ``projetos/``, ``playbooks/``, ``_fontes/``, ``_arquivados/``, ``00_inbox/``.
3. Crie ``wiki/_schema.md`` documentando o propósito de cada pasta (`clientes` = card.md superficial + deep.md tático; `padroes` = como fazemos as coisas; `decisoese` = ADRs datados; etc.).
4. Crie ``wiki/index.md`` como página raiz com wikilinks ``[[`` pra cada subpasta.
5. Adicione ao meu CLAUDE.md as regras: (a) antes de tarefa com cliente, ler ``wiki/clientes//card.md``; (b) após call ou doc novo, perguntar se quer ingerir na wiki; (c) clientes legacy podem usar ``wiki/clientes/.md`` único.
6. Sugira instalar Obsidian e abrir o vault apontando pra ``wiki/`` (opcional mas recomendado).

NÃO crie páginas de cliente automaticamente – espere eu pedir.

Cuidado: wiki só funciona se você ingerir conhecimento de verdade. Página vazia não ajuda. A regra que eu sigo: depois de toda call ou doc novo, pergunto "isso vira wiki?". Maioria não — mas as que viram, valem ouro 6 meses depois.

MemPalace — memória *semântica* entre conversas

MCP server que combina Knowledge Graph (triples sobre entidades nomeadas — cliente, projeto, pessoa) e busca vetorial. O Claude recupera percepções que você teve em sessões anteriores, mesmo que nem lembre de ter falado. É a primeira camada do pipeline **percepção** → **ação** → **consolidação**: fragmentos vivem aqui, e só os durables sobem pra wiki.

TIPO	PRÉ-REQUISITOS	QUANDO NÃO USAR
MCP server (local)	Python 3.11+ · ChromaDB	Decisão já consolidada — vai pra wiki/decisoese/

PROMPT PRA COLAR NO CLAUDE CODE

Instale o MemPalace MCP server (gabrielspricigo/mempalace ou repo equivalente).

Passos:

1. Clone o repo do MemPalace para `~/code/mempalace` (peça URL do repo se não souber).
2. Crie venv: `python3 -m venv ~/code/mempalace/.venv` e ative.
3. Instale dependências: `pip install -r requirements.txt` (deve incluir chromadb, sentence-transformers).
4. Configure o MCP em `~/.claude/.mcp.json` (ou similar do meu Claude Code) — adicione mempalace como servidor MCP apontando pro `~/code/mempalace/.venv/bin/python` (caminho absoluto, NÃO `python3` bare).
5. Teste: reinicie Claude Code e veja se `mempalace\_status` aparece nas tools disponíveis.
6. Adicione ao meu CLAUDE.md a regra: default = `mempalace\_kg\_add` (triple), não drawer. Drawer só pra blob livre. Sempre criar room específica (NUNCA `general`).
7. ⚠ NÃO rode `mempalace mine .` na raiz do repo — gera milhares de chunks lixo e quebra a busca. Se quiser mining, alvo específico (ex: só `wiki/`).

Se ChromaDB pedir versão específica, prefira a stable mais recente.

Padrão que eu sigo: ao mencionar entidade nomeada (cliente, projeto, pessoa), rodar mempalace_kg_query em paralelo com a busca semântica. Side quests aparecem com predicate side_quest_* — historia recuperável meses depois.

/done — fechar etapa, *calibrar* skill, comemorar

O ritual de fim de toda etapa. Faz três coisas numa passada: registra o que foi entregue, varre as skills usadas na sessão perguntando se alguma precisa ajuste (auto-improvement Nível 4, a skill aprende com você) e — se você usa ClickUp — fecha a task. Quem roda /done regular tem skills que evoluem sozinhas.

TIPO	PRÉ-REQUISITOS	QUANDO NÃO USAR
Slash command	Nenhum (ClickUp opcional)	No meio da execução — só no fim

PROMPT PRA COLAR NO CLAUDE CODE

Instale a skill ``/done`` em `~/ .claude/skills/done/SKILL.md``. Estrutura:

- FRONTMATTER: name=done, description="Ritual de fim de etapa: comemoração + calibração de skills (Nível 4 auto-improvement) + bridge ClickUp opcional."
- TRÊS ESCOPOS (detecte pelo contexto da sessão):
 - fim de etapa → comemoração curta + calibração
 - fim de task → comemoração + calibração + bridge ClickUp (se configurado)
 - fim de projeto → retrospectiva curta + calibração + bridge
- CALIBRAÇÃO NÍVEL 4 (parte mais importante – é o coração da skill):
 - Liste todas as skills invocadas nesta sessão (extraia do contexto).
 - Pra cada uma, pergunte: "Algo a calibrar em ?"
 - Se sim, pegue a regra curta (≤200 chars) e adicione no SKILL.md daquela skill numa seção "Regras vivas" (bullet com data de hoje, cap 10 – mais antigo sai pra arquivo).
- BRIDGE CLICKUP (pergunte ao usuário ANTES de configurar):
 - "Você usa ClickUp?" – se NÃO, pule essa parte, /done roda standalone.
 - Se SIM: peça o API token (salva em `~/ .config/.env` como `CLICKUP_API_TOKEN`) e os IDs das listas principais. Ao concluir uma task, /done pergunta "fecha no ClickUp?" e executa via API.
- TESTE: rode ``/done`` agora – deve listar esta instalação como sessão e perguntar o que calibrar.

Por que é o último: as outras 5 skills te dão capacidade. /done faz elas *evolúrem* — cada calibração vira regra viva, sua versão pessoal da skill.

PRÓXIMO PASSO · MENTORIA

Depois de instalar, *abra uma sessão* comigo

Antes da nossa próxima call, valide o setup com o checklist abaixo. Se algum item travar, manda print na thread da mentoria — a gente desbloqueia ali mesmo.

CHECKLIST DE VALIDAÇÃO

- 01 **RTK** respondendo: `rtk --version` mostra versão e `rtk gain` abre sem erro.
- 02 **OpenWispr** ditando: segurar `fn`, falar uma frase, texto aparece no editor ativo.
- 03 **agent-browser** navegando: `agent-browser snapshot https://example.com` retorna árvore com refs `@eN`.
- 04 **Wiki** esqueleto: pasta `wiki/` existe no repo com subpastas mínimas e `_schema.md` preenchido.
- 05 **MemPalace** ativo: `mempalace_status` aparece nas tools do Claude Code após restart.
- 06 **/done** instalado: `/done` abre o ritual e lista as skills usadas na sessão pra calibrar.

Esse kit não te transforma em mim. Te dá o mesmo solo de onde eu construo. O que cresce nele é seu.

— G. P.